POLITECNICO DI BARI

DEI DEPARTMENT

MASTER'S DEGREE IN AUTOMATION ENGINEERING

Optimization & Control

Prof. Eng. Agostino M. Mangini

Project:

Goal-Oriented Pareto Optimization of a 2DOF Manipulator

Student:
Davide Tonti

ACADEMIC YEAR 2025-2026

Abstract

This work extends the methodology presented by Chaudhary et al. in 'Design of Serial Link Manipulator with Uncertainties using Interval Method' [2]. While the original study focuses on analyzing the achievement of a goal using grid search and interval methods , utilizing two pseudo-independent optimizations of physical parameters, this work introduces a multi-objective optimization framework paired with a dynamic control scheme and a Simscape-based physical model. By transitioning from a static design focus to a multi-cost approach, we provide a comprehensive analysis of the manipulator's performance from a dynamic perspective. The proposed model evaluates the trade-offs between Dynamic Manipulability, the Local Conditioning Index (LCI), and target proximity, offering a more reliable and robust representation of real-world operating conditions.

Contents

1 Introduction **1**

2 Simscape Model **2**

3 Simulink Control **4**

4 Performance Measures **6**

5 Simulation-In-The-Loop Pareto Front **7**

6 Results **8**
6.1 Comparison of the Pareto front without proximity constraint 8
6.2 Comparison of the Pareto front with proximity constraint 12
6.3 Considerations 16

7 Conclusions and future development **18**

References **19**

1 Introduction

The optimization work with simulation-in-the-loop for is divided into three main parts, in order:

- Modeling the manipulator on Simscape multibody [3]
- Modeling the control system on Simulink [4]
- Optimizing the physical parameters with a multi-objective function.

The simulation was designed so that, with just a few simple objects on Simscape, it is possible to obtain a graphical and intuitive representation of the model, link masses, inertia matrices if needed, and set the gravity acting on the solids. In essence, it is possible to recreate the manipulator model without modeling it mathematically, but simply by joining blocks and joints together and focusing on inputs and outputs, with an implicit degree of mathematical accuracy that is often superior to that calculated by hand.
The control system is essential when it comes to manipulators and robotics in general. Without even a simple control system, everything we talk about remains interesting, but only clear and solid from an ideal mathematical point of view. In this case, we wanted to recreate a closed-loop control system in Simulink using a block system, which is well known in the field of automation.
Finally, optimization was designed with the same goals as the original paper, but in a more refined and intuitive way, with a Pareto front where each set of decision variables is sent to the simulation and tested.

Each point is discussed in greater detail in the following paragraphs.

2 Simscape Model

The Simscape model is a planar manipulator on the X-Z plane with gravity acting on Z. The links consist of hollow cylinders with lengths l1 and l2 and radii R1 and R2, and there is a payload with a mass of 5 kg. The structure was reconstructed in accordance with the parameters specified in the paper.
With the exception of the "solver configuration," "world frame," and "mechanical configuration" blocks configured with gravity on the z-axis, which are standard Simscape blocks, the following were used: rigid transform block, revolved solid, revolute joint, spherical solid, and inertia sensor.

The *revolute joint* is used to set the rotation for the two joints and has the following configurations:

- actuation: torque provided by input, motion automatically computed
- initial configuration that establishes how the arm will be configured (rest position)
- the upper and lower constraints on rotation are set directly here
- the position and speed of the joint are obtained directly from here (it is therefore assumed that there will be sensors capable of determining the position and rotation of the joints at each sampling, such as an encoder, for example)

The *actuation* configuration is used to control torque and therefore current in such a way that the actuators (motors) are assumed to be real and capable of delivering torque according to the appropriate trajectory, where gravity has an effect. Otherwise, the system will follow the trajectory exactly and the weight of the payload will be influential. The latter approach is only used to experiment with kinematic trajectory control, but this is not the case here.
The *revolved solid* used to specify that the cylinder is hollow with radii defined by outer radius and outer radius - constant thickness; it's specified also the density of the material.
The *spherical solid* is used as a payload with a specified mass or, like the brick solid, for aesthetic purposes in binding the various solids together.
Finally the *inertia sensors* to export the masses and the centre of gravity to the frame where they are connected. A frame is also placed to indicate the presence of the sensor on the simulator itself. There are two: the first is related to the link 1, the second is related to the second link and payload, excluding the link 1. This is important because the Centre of Gravity is shifted in the second frame beacuse of the payload, so it's not placed exactly in the middle of the link and have impact on the control.
When modeling, it was important to specify the *orientation of the frames* for each solid, because this also determines where and how gravity acts. As already mentioned, the planar manipulator is on the X-Z plane with gravity acting on Z. To orient the frames correctly, a rigid transform block was used, and a custom frame orientation was used for each link and payload. Next, the complete Simscape modeling (fig. 1) and the resulting manipulator with the frames in the initial rest position (fig. 2).

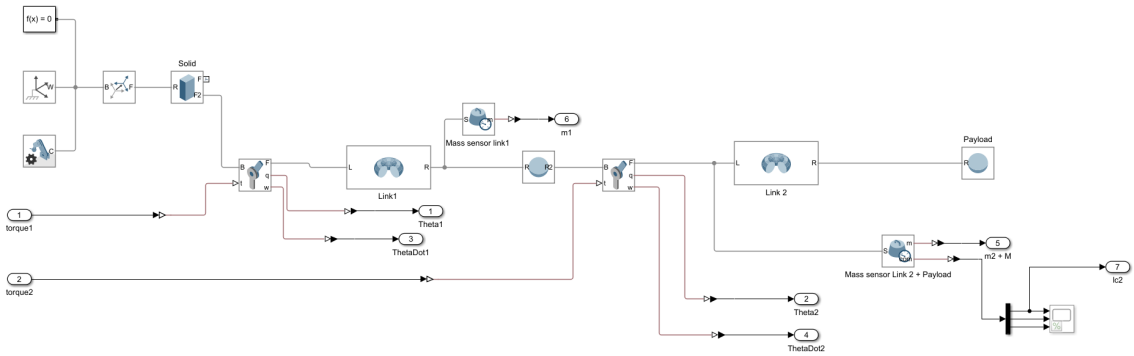

Figure 1: Simscape Model

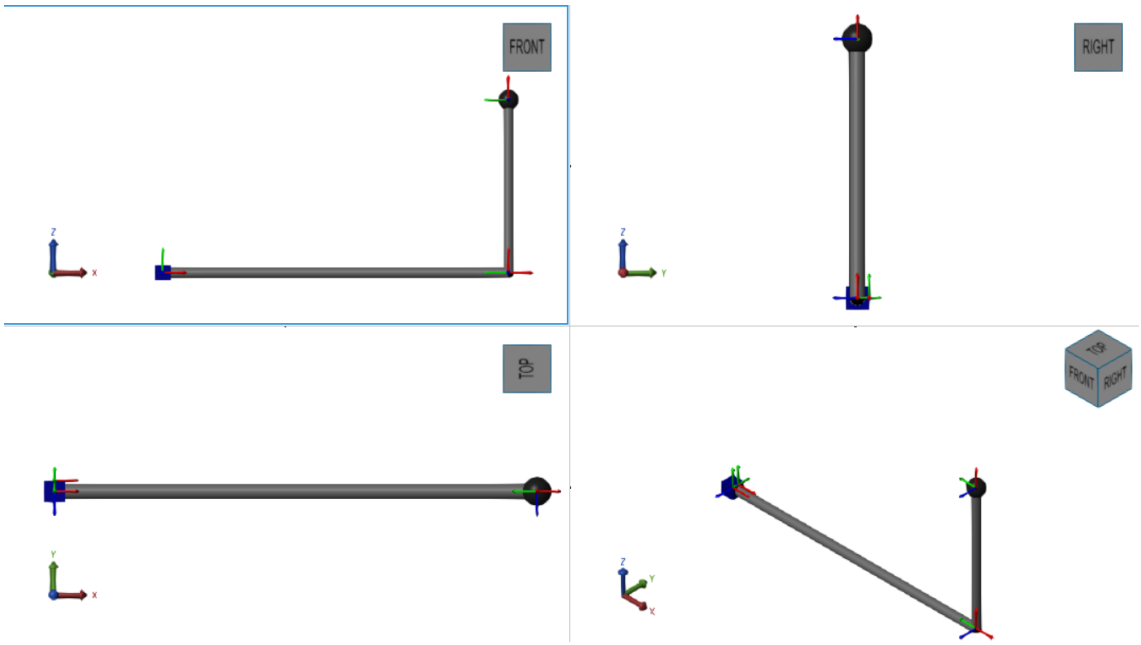

Figure 2: Manipulator View

3 Simulink Control

The closed-loop control system was developed in Simulink. Several subsystems were used to group all blocks by purpose.
The subsystems and MATLAB function blocks used are for:

- Cubic trajectory
- Inverse kinematics
- Mechanical model (Simscape)
- Gravity compensation
- Performance measures
- Direct kinematics
- X-Z error computation

A control system taken from the literature [5] was developed: *PD control with gravity compensation in the joint space* (fig.3). This control system allows for feedback of the non-linear contribution of gravity, which is strongly present in this system, leaving the PD to control the rest of the system in a linear manner. The system will therefore not only hold up with the PD in an agnostic manner with very high gain values, but will also be indirectly aware of gravity: in a singular condition (arm fully extended), the PD will be zero because gravity is completely balanced by the structure itself and the arm does not need to be controlled; otherwise, the PD will have to balance just enough to follow the trajectory and the then back to zero in the target position in equilibrium.
For this reason, the transformation from the operating space of the end-effector (payload) to the joint space with the respective angles of the two desired input joints was first performed using the inverse kinematics block, from which the difference with the actual angles of the structure was calculated and an error was calculated, which was given to the PD controllers and added to the gravity compensation block. The angles per joint and the speeds are taken from the structure to close the loop and perform the dynamic performance measurements with the other constant parameters. Finally, direct kinematics is also performed to calculate the actual payload position, from which the error with respect to the input position in the operating space is calculated. To take into account a real, non-instantaneous movement from an initial point to the target, a cubic polynomial was introduced for the payload trajectory with zero initial and final velocities. The functions in the blocks are those of a classic planar manipulator, but in some cases they have been adapted because it is rotated on the X-Z plane.
The figure 4 shows the corresponding complete Simulink diagram.

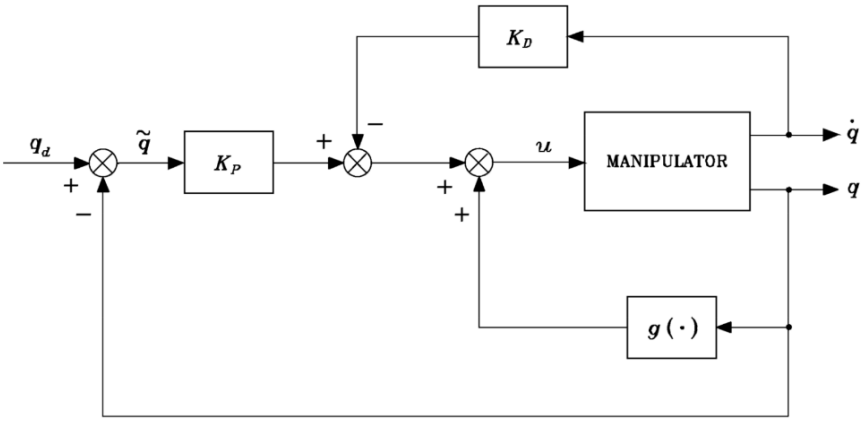

Figure 3: PD with gravity compensation scheme

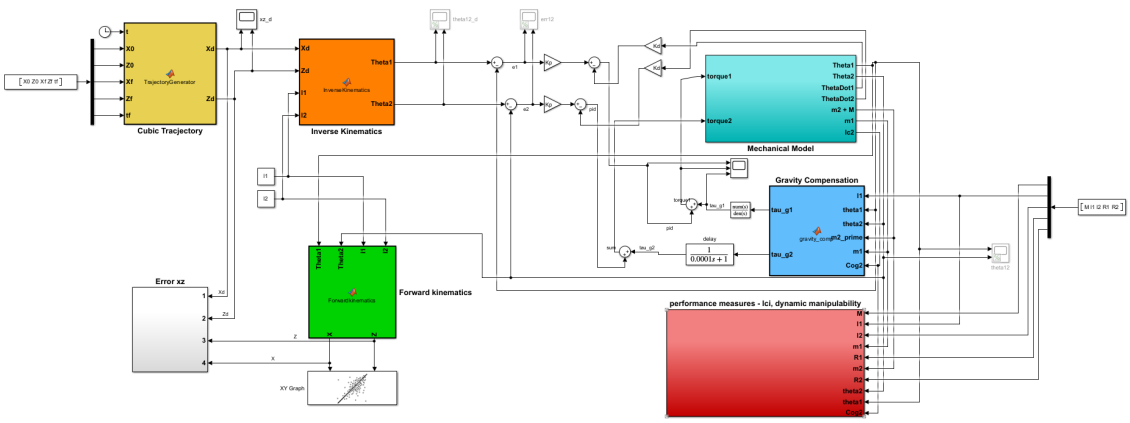

Figure 4: Simulink-Simscape model

4 Performance Measures

The performance measures from the paper were used, with one modification to the conditional index. The original paper uses the global conditioning index to obtain a general measurement across the entire workspace. This is useful in principle, but given that the model, control, and trajectory are available, in the event of having to perform a specific task, it is advisable to calculate the local conditioning index for each point of the trajectory, which has replaced it.
Let's briefly recap why these Performance measures are used, citing the paper [6] :

"The concept of a *dynamic manipulability* measure quantifies the manipulating ability of a robot's endeffector with consideration of its arm dynamics. This measure is an extension of the simple manipulability index μ, which is a kinematic performance metric introduced by Yoshikawa. Dynamic manipulability index measures the ability of the manipulator to generate acceleration based on a joint driving force [...]".

"The *condition number* is a good measure of the manipulator's distance singularity and kinematic accuracy. Unlike the minimum singular value (MSV), the conditioning number only considers two directions of motion for the end-effector, the most difficult and the most easiest direction of motion, ignoring the rest [...]".
For further information, please refer to the paper cited in the reference.

A third measure has also been added, namely *proximity to the goal*. In fact, by changing the lengths of the arms, the goal will result in a variable error that depends, when fully operational, precisely on a sizing error. This error is calculated on both the x and z axes and has been unified by adding them together in absolute value once the control regime has been reached. In the original paper, there was only a check on the kinematics with nominal values and a *tolerance* using the interval method. With the control approach, this approach was replaced with a more realistic one, considering that the tolerance of the material, being very small, is naturally corrected by the controller, reducing the analysis to the absolute values of the parameters.

5 Simulation-In-The-Loop Pareto Front

Once the modeling, control, and performance metrics have been defined, we can construct a Pareto front of solutions based on the mechanical design variables. By definition, the Pareto optimal solutions are not dominated by other admissible solutions; there are no admissible solutions that are better than a Pareto optimal solution with respect to at least one objective.
In practice, it is best to treat multiple objective functions together if possible, if they are competitive with each other, and evaluate in post-processing which set of solutions is most suitable for the set objective. In the case of a manipulator, tolerance is certainly more important than the target and therefore the other two objective functions.
In particular, we chose to create a front with the performance measures described in the previous paragraph subject to the constraints inherent in the Simscape model itself, namely total length constraints and upper and lower limits.

To do this, Matlab's *multi-objective genetic algorithm* and Matlab's *paretosearch* were used, passing them the constraints which, iteratively, searched for the lengths and radii relative to the links, passed them to the Simulink and Simscape models, started the simulation, and took the cost function values from it.

It proceeded to:

- Maximize the average dynamic manipulability
- Maximize the minimum worst-case LCI
- Minimize the error with respect to the target

It is possible to dynamically see how the simulator that tests the various configurations starts up graphically (not recommended except for testing, as it consumes part of the computation to start the interface). Optimization takes some time. On my PC, it took 50 minutes for the genetic algorithm and 10 minutes for the Pereto search, but this varies depending on the hardware performance on which it is run. Parallelization was not used in the functions themselves due to a parameter concurrency issue in the simulator.

Taking directly from the Matlab documentation [7], : "gamultiobj uses a controlled, elitist genetic algorithm (a variant of NSGA-II). An elitist GA always favors individuals with better fitness value (rank). A controlled elitist GA also favors individuals that can help increase the diversity of the population even if they have a lower fitness value. It is important to maintain the diversity of population for convergence to an optimal Pareto front. Diversity is maintained by controlling the elite members of the population as the algorithm progresses.", so the gamultiobj algorithm balances the selection of high-performing individuals with the preservation of unique, diverse candidates to ensure the entire Pareto front is explored without getting stuck in a single, repetitive solution.
And then [8]: "The paretosearch algorithm uses pattern search on a set of points to search iteratively for nondominated points.[...] The pattern search satisfies all bounds and linear constraints at each iteration. Theoretically, the algorithm converges to points near the true Pareto front. "

It is possible to change the tolerance constraint in the script. If left unrestricted, this allows for better exploration of the solution space, but on the other hand, if the constraint is strict, there will be few points to select. Conversely, if the constraint is imposed, there will be more points to select but less exploration. For the sake of completeness, I simulated both cases.

6 Results

This section presents the results of the different Pareto fronts, analyzed with and without tolerance constraints, with the optimization parameters in the table 1 for both algorithms. Obviously, for each of these points, it is possible to see the values of the objective functions and solutions of length and radii associated with the manipulator. These are saved and returned in the output once optimization is complete on the terminal. For each simulation, rotation *constraints*, maximum length sums, upper and lower bounds on mechanical parameters are imposed on the model.

Table 1: Configuration for gamultiobj e paretosearch

Param	gamultiobj	paretosearch
`PopulationSize`	60	-
`MaxGenerations`	50	-
`ParetoSetSize`	-	20
`MaxIterations`	-	50

6.1 Comparison of the Pareto front without proximity constraint

The following figure 5 - 10 shows the relative Pareto fronts without proximity constraints. The tables 2 - 3 show the results associated with the plots.

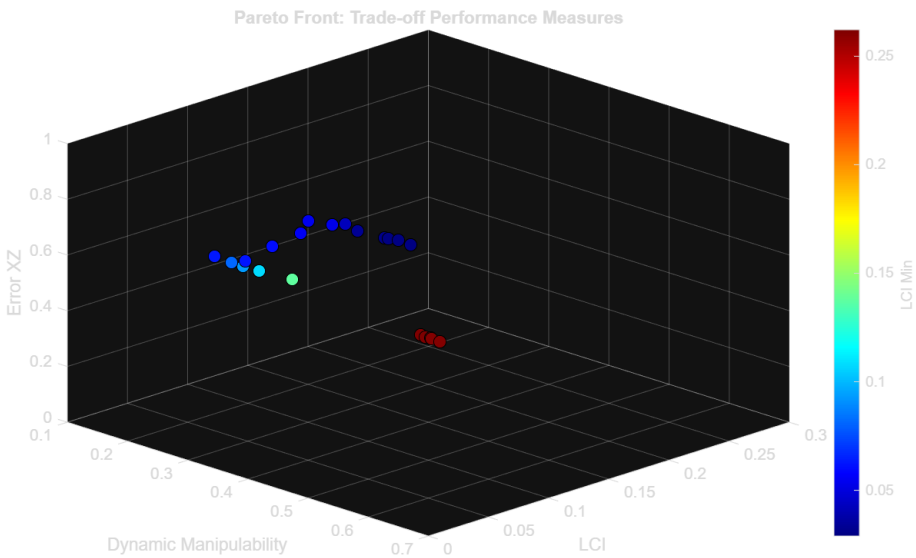

Figure 5: Gamultiobj with complete exploration

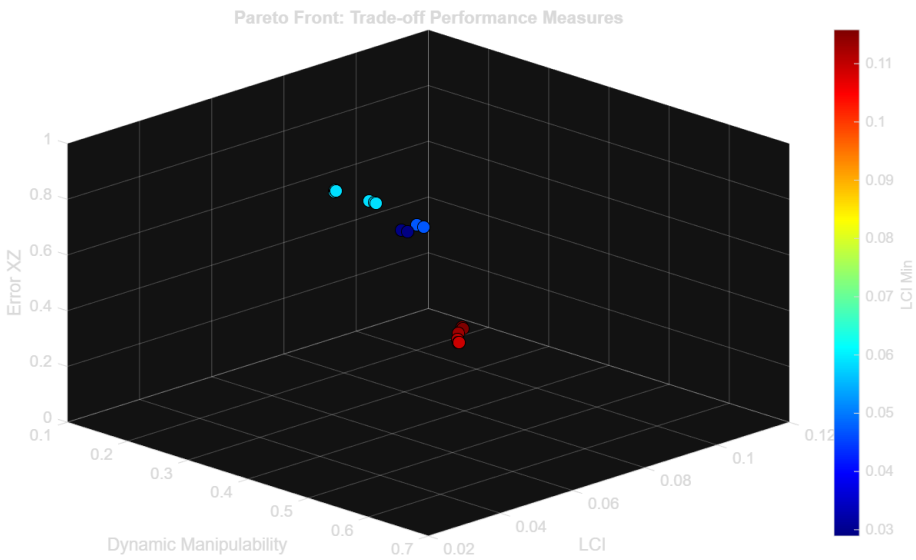

Figure 6: Paretosearch with complete exploration

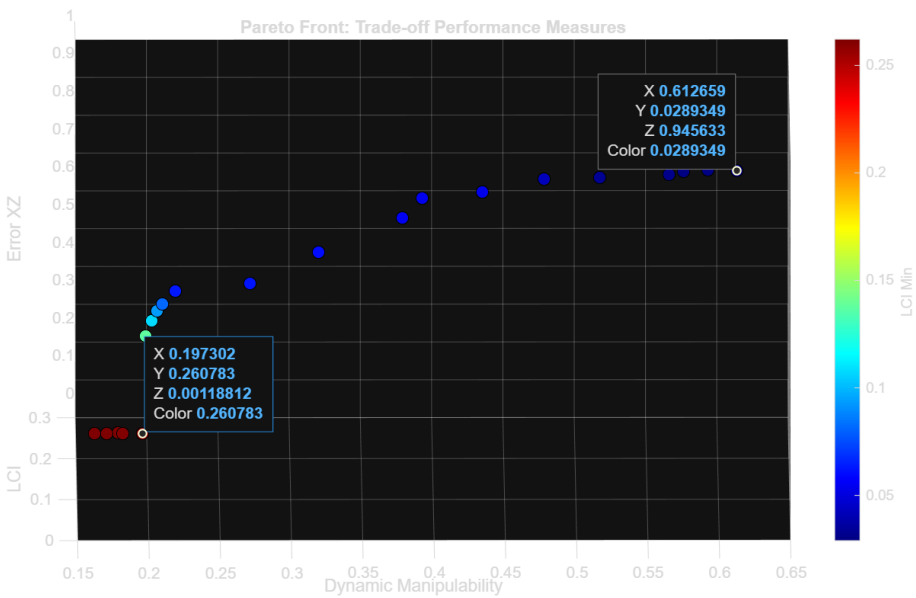

Figure 7: Gamultiobj with complete exploration. error XZ-DM view

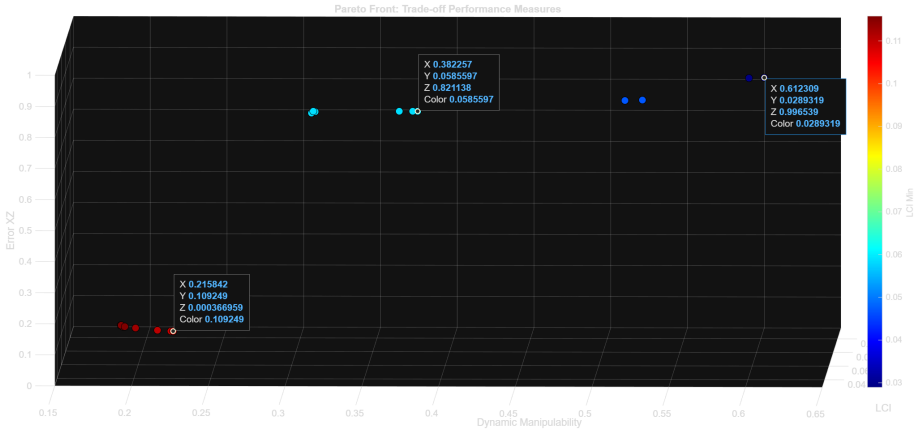

Figure 8: Paretosearch with complete exploration. error XZ-DM view

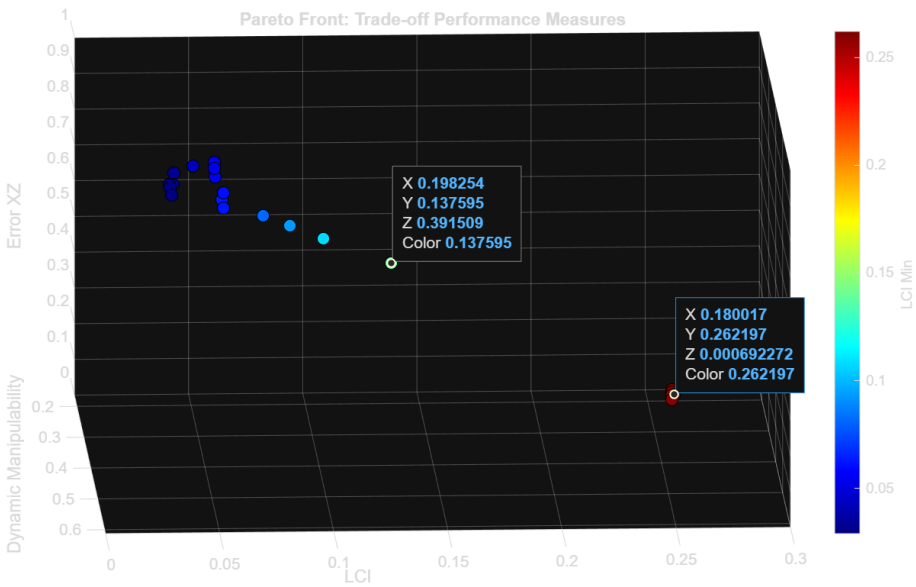

Figure 9: Gamultiobj with complete exploration. error XZ-LCI view

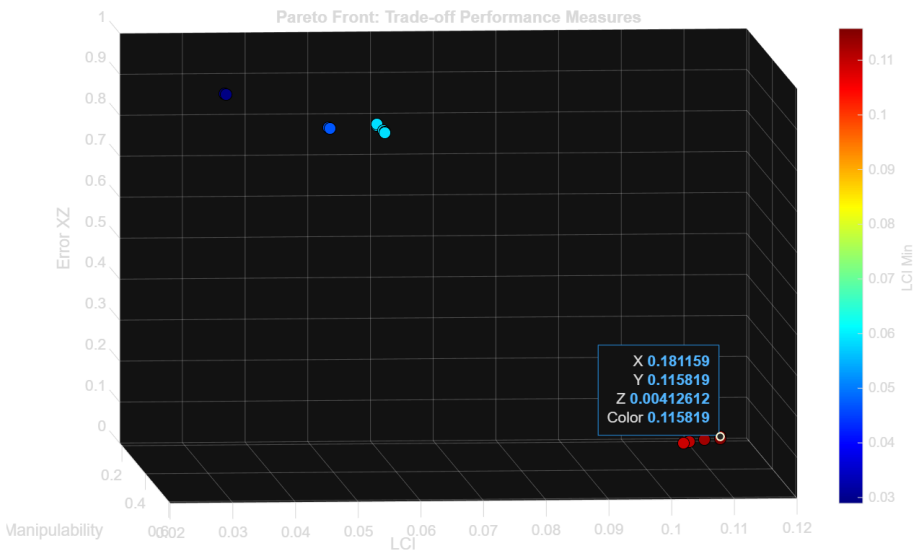

Figure 10: Paretosearch with complete exploration. error XZ-LCI view

Table 2: gamultiobj table with no proximity contraint

DM	LCI	Errorxz	L1 (m)	L2 (m)	R1 (m)	R2 (m)
0.1722	0.2614	0.0000	0.6039	0.3961	0.0186	0.0112
0.1983	0.1376	0.3915	0.2966	0.7034	0.0190	0.0148
0.6127	0.0289	0.9456	0.1667	0.8333	0.0213	0.0161
0.2187	0.0630	0.5913	0.2695	0.7305	0.0210	0.0160
0.5923	0.0289	0.9489	0.1667	0.8333	0.0191	0.0161
0.2027	0.1079	0.4654	0.2851	0.7149	0.0209	0.0158
0.1834	0.2610	0.0014	0.6619	0.3381	0.0475	0.0103
0.5752	0.0289	0.9439	0.1667	0.8333	0.0195	0.0204
0.1800	0.2622	0.0007	0.6382	0.3623	0.0295	0.0129
0.2101	0.0812	0.5378	0.2756	0.7244	0.0210	0.0159
0.1973	0.2608	0.0012	0.6664	0.3336	0.0189	0.0114
0.3917	0.0544	0.8455	0.2496	0.7504	0.0202	0.0154
0.3195	0.0607	0.6968	0.2655	0.7345	0.0208	0.0160
0.2061	0.0930	0.5061	0.2797	0.7203	0.0201	0.0151
0.3782	0.0550	0.7934	0.2514	0.7486	0.0262	0.0152
0.4776	0.0423	0.9093	0.2145	0.7855	0.0198	0.0161
0.5648	0.0312	0.9346	0.1756	0.8244	0.0218	0.0160
0.2713	0.0622	0.6120	0.2692	0.7308	0.0209	0.0167
0.5161	0.0330	0.9245	0.1824	0.8176	0.0218	0.0237
0.4341	0.0531	0.8640	0.2461	0.7539	0.0209	0.0154
0.1637	0.2616	0.0000	0.5995	0.4014	0.0439	0.0150

Table 3: paretosearch table with no proximity contraint

DM	LCI	Errorxz	L1 (m)	L2 (m)	R1 (m)	R2 (m)
0.6123	0.0289	0.9965	0.1667	0.8333	0.0100	0.0100
0.5188	0.0471	0.8826	0.2292	0.7708	0.0100	0.0100
0.1908	0.1131	0.0014	0.6042	0.3958	0.0100	0.0100
0.3154	0.0585	0.8198	0.2604	0.7396	0.0500	0.0100
0.3823	0.0586	0.8211	0.2604	0.7396	0.0100	0.0100
0.3704	0.0586	0.8218	0.2604	0.7396	0.0100	0.0100
0.1812	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.1824	0.1158	0.0005	0.4792	0.5208	0.0100	0.0100
0.3792	0.0586	0.8221	0.2604	0.7396	0.0100	0.0100
0.6021	0.0289	0.9951	0.1667	0.8333	0.0100	0.0100
0.2158	0.1092	0.0004	0.6667	0.3333	0.0100	0.0100
0.5304	0.0470	0.8835	0.2292	0.7708	0.0100	0.0100
0.1834	0.1158	0.0005	0.4792	0.5208	0.0100	0.0100
0.1810	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.3144	0.0586	0.8203	0.2604	0.7396	0.0500	0.0100
0.1809	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.3132	0.0586	0.8156	0.2604	0.7396	0.0500	0.0100
0.2144	0.1092	0.0004	0.6667	0.3333	0.0100	0.0100
0.1908	0.1131	0.0014	0.6042	0.3958	0.0100	0.0100
0.2056	0.1104	0.0006	0.6510	0.3490	0.0100	0.0100

6.2 Comparison of the Pareto front with proximity constraint

Below (fig. 11 - 16) is an overview of the results obtained by imposing a proximity constraint (sum of the absolute value between error on x and z) of less than 0.16m.
The tables 4 - 5 show the results associated with the plots.

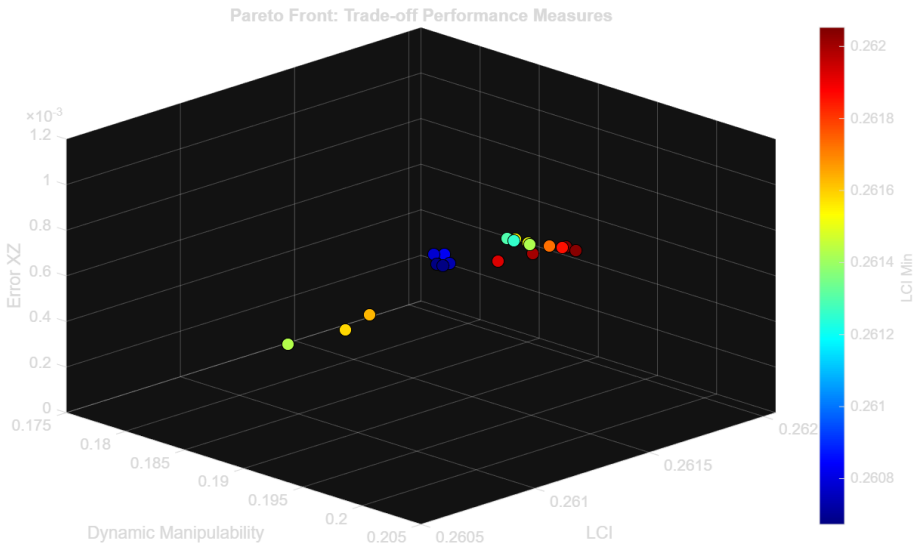

Figure 11: Gamultiobj with proximity contraint

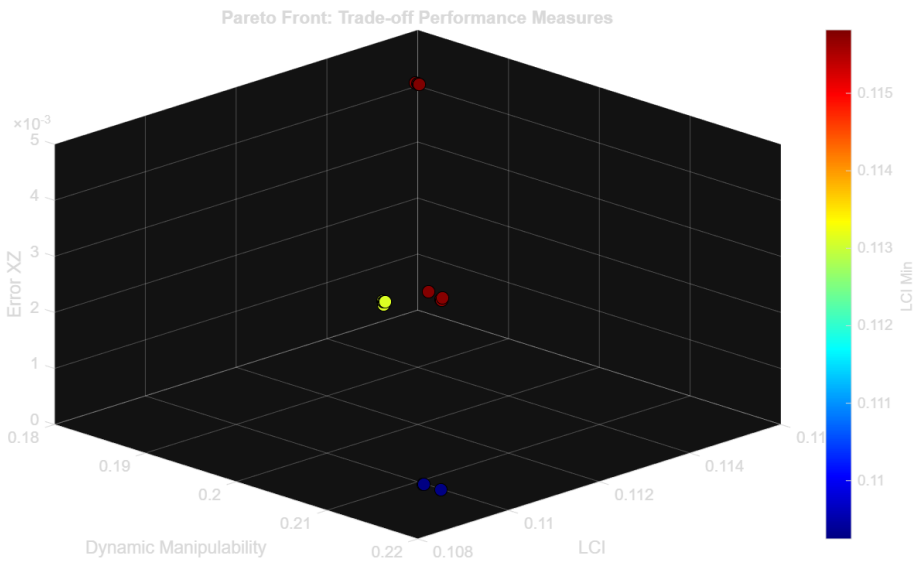

Figure 12: Paretosearch with proximity contraint

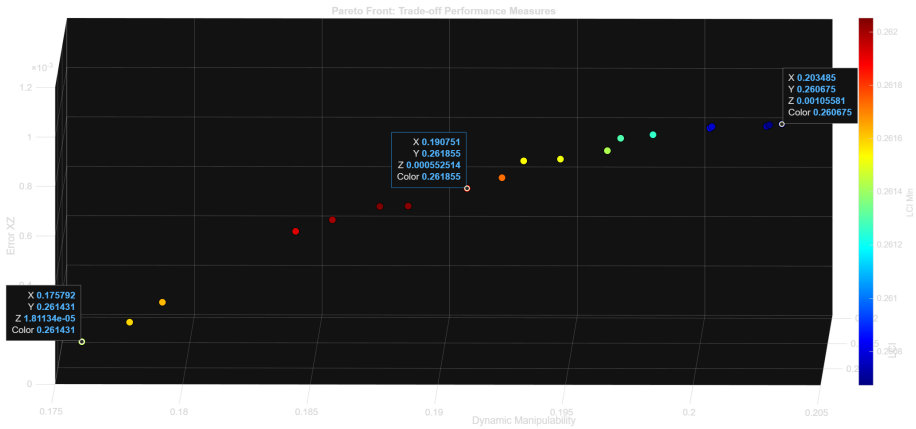

Figure 13: Gamultiobj with proximity contraint. error XZ-DM view

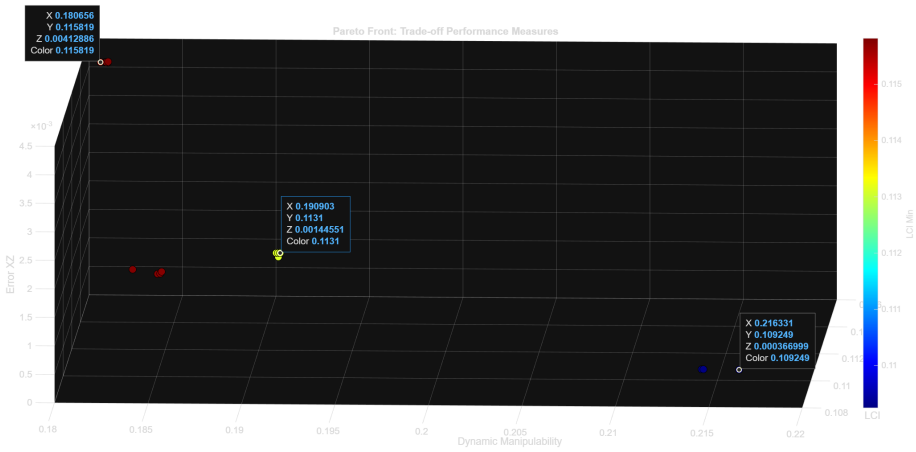

Figure 14: Paretosearch with proximity contraint. error XZ-DM view

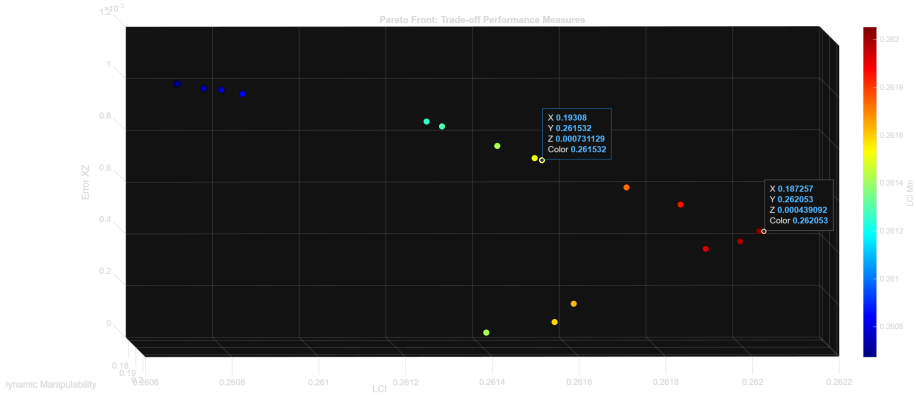

Figure 15: Gamultiobj with proximity contraint. error XZ-LCI view

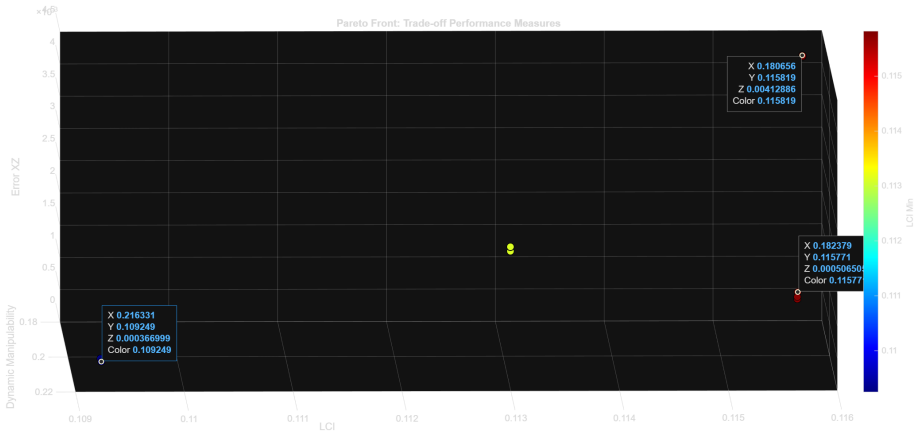

Figure 16: Paretosearch with proximity contraint. error XZ-LCI view

Table 4: gamultiobj table with proximity contraint

DM	LCI	Errorxz	L1 (m)	L2 (m)	R1 (m)	R2 (m)
0.1873	0.2621	0.0004	0.6346	0.3654	0.0102	0.0101
0.2035	0.2607	0.0011	0.6666	0.3334	0.0100	0.0101
0.1758	0.2614	0.0000	0.6049	0.3951	0.0163	0.0106
0.1964	0.2614	0.0008	0.6545	0.3455	0.0105	0.0102
0.1922	0.2617	0.0006	0.6452	0.3548	0.0103	0.0103
0.1884	0.2620	0.0004	0.6351	0.3649	0.0102	0.0101
0.1982	0.2613	0.0009	0.6589	0.3411	0.0107	0.0102
0.2029	0.2607	0.0010	0.6658	0.3342	0.0102	0.0101
0.1776	0.2616	0.0001	0.6092	0.3908	0.0117	0.0102
0.1945	0.2615	0.0007	0.6518	0.3482	0.0104	0.0102
0.2007	0.2608	0.0010	0.6651	0.3349	0.0102	0.0101
0.2035	0.2607	0.0011	0.6666	0.3334	0.0100	0.0101
0.1908	0.2619	0.0006	0.6412	0.3588	0.0107	0.0103
0.2030	0.2607	0.0011	0.6667	0.3333	0.0100	0.0101
0.2006	0.2608	0.0010	0.6641	0.3359	0.0102	0.0103
0.1758	0.2614	0.0000	0.6049	0.3951	0.0163	0.0106
0.1854	0.2620	0.0004	0.6316	0.3684	0.0111	0.0103
0.1970	0.2613	0.0009	0.6583	0.3417	0.0104	0.0101
0.1840	0.2619	0.0004	0.6281	0.3719	0.0139	0.0104
0.1931	0.2615	0.0007	0.6511	0.3489	0.0107	0.0102
0.1789	0.2616	0.0001	0.6127	0.3873	0.0163	0.0106

Table 5: paretosearch table with proximity contraint

DM	LCI	Errorxz	L1 (m)	L2 (m)	R1 (m)	R2 (m)
0.1837	0.1158	0.0004	0.4792	0.5208	0.0100	0.0100
0.1810	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.2163	0.1092	0.0004	0.6667	0.3333	0.0100	0.0100
0.1909	0.1131	0.0014	0.6042	0.3958	0.0100	0.0100
0.2144	0.1092	0.0004	0.6667	0.3333	0.0100	0.0100
0.1810	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.1809	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.1839	0.1158	0.0005	0.4792	0.5208	0.0100	0.0100
0.1807	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.1908	0.1131	0.0014	0.6042	0.3958	0.0100	0.0100
0.1824	0.1158	0.0005	0.4792	0.5208	0.0100	0.0100
0.2143	0.1092	0.0004	0.6667	0.3333	0.0100	0.0100
0.2143	0.1092	0.0004	0.6667	0.3333	0.0100	0.0100
0.1908	0.1131	0.0014	0.6042	0.3958	0.0100	0.0100
0.1810	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.1809	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.1907	0.1131	0.0014	0.6042	0.3958	0.0100	0.0100
0.1908	0.1131	0.0014	0.6042	0.3958	0.0100	0.0100
0.1811	0.1158	0.0041	0.5104	0.4896	0.0100	0.0100
0.1839	0.1158	0.0004	0.4792	0.5208	0.0100	0.0100

6.3 Considerations

We note how the genetic algorithm tends to create a front that seeks to cover as much space as possible, while paretosearch aggregates into subgroups that the algorithm considers most promising for the same number of solutions.

LCI values seems low compared to the value of GCI in the original paper; this is for various reasons:

- the local index used in relation to the GCI takes into account the worst case scenario in terms of manipulator dexterity and not the maximum or average value as done in the paper. In fact, by analyzing one of the Pareto points, we can see the actual trend of the LCI for each point of the trajectory (fig. 17)
- it depends on the mass of the payload, the structure itself, and the calibrated PIDs of the motors without "exceeding" and distorting the real approach. In fact, only by setting the payload to 0, the trend of LCI (and DM) rise

The value obtained from this work shows that these are not just mathematical values that depend solely on kinematics but also on the aspects addressed: control, dynamics, and trajectory.

Note: Unlike LCI, the value of dynamic manipulability is not normalized and therefore does not have a true range (example fig. 18), the important thing is to maximize it on average.
With regard to the proximity constraint, even with a moderate tolerance constraint, the optimization still chose a proximity of less than 0.005, preferring configurations in which the first link approaches its upper bound with an average link ratio of 1.6 - 1.75 in reference to the original paper and an average radius ratio of 1.
Compared to the case without a minimum proximity constraint, the lengths, or even more so the radii, have a smaller numerical range.

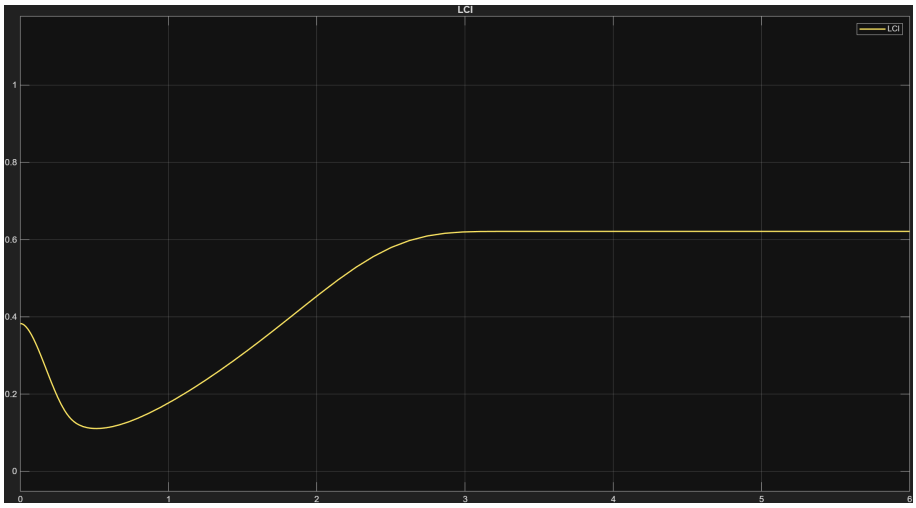

Figure 17: LCI trend

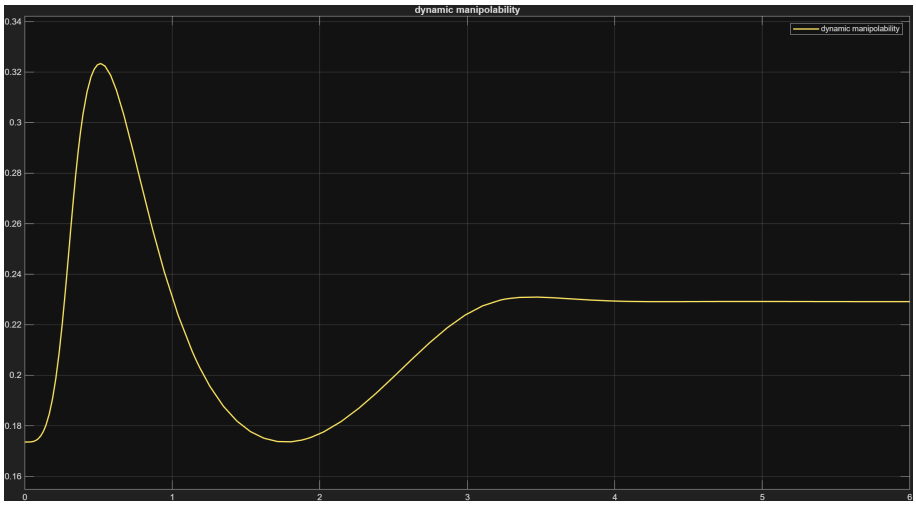

Figure 18: DM trend

7 Conclusions and future development

The work conducted in this project has shown highly promising results, proposing and validating a paradigm shift compared to the theoretical approach analyzed in the original paper.
By replacing the static analysis based on the interval method with a simulation-in-the-loop optimization framework, it was possible to directly interface the mechanical design with real dynamic control logic. The combined use of multibody modeling and a PD control with gravity compensation allowed for the observation of the system's actual behavior, moving away from ideal simplifications. The analysis highlighted how a planar manipulator with only two degrees of freedom presents intrinsic difficulties, which become particularly evident when transitioning from a purely kinematic domain to a physical one. In a simulated environment that accounts for gravity acting on the Z-axis and the real inertia of the materials, structural stresses and actuation bottlenecks cannot be ignored. The need to balance competing objectives such as maximizing Dynamic Manipulability and the Local Conditioning Index against minimizing the target proximity error, clearly brought out the challenges associated with adopting real world parameters and materials. However, despite the introduction of these complex physical non-linearities, the obtained results maintain a solid consistency with the estimates derived from purely mathematical works, showing deviations that are limited and entirely physiological.

This divergence between the ideal model and the tested physio-dynamic one can be primarily attributed to two critical operational factors. First, the significant shift in the center of gravity of the second link, caused by the interaction between the structure and the addition of the payload. Second, the actual joint dynamics, where operational and torque limits influence the motor's ability to seamlessly compensate for errors while tracking the trajectory.The comparative use of two distinct multi-objective solvers, gamultiobj and paretosearch, further enriched the analysis by revealing different exploration strategies of the Pareto front. While the genetic algorithm (gamultiobj) demonstrated a broader exploration of the solution space to maintain population diversity, the pattern search algorithm (paretosearch) tended to cluster solutions into subgroups deemed more promising.

This comparison confirmed that the choice of the optimization algorithm directly impacts how the trade-offs between mechanical design variables are mapped. Ultimately, this study demonstrates that a truly robust design of a robotic manipulator cannot be achieved through isolated mechanical sizing. Instead, it requires an integrated co-design strategy. The goal-oriented optimization of physical parameters, such as link lengths and mass distribution, must be concurrently developed with the dynamic control architecture. Only by intertwining structural sizing with dynamic calibration can the system guarantee reliable performance under tangible operating conditions.

As for future developments, we could make the model more complex by specifying the motor type (e.g., brushless motor) for each joint, along with its torque limits, or we could increase the complexity of the manipulator,such as by creating a 6-DOF version, to evaluate more practical scenarios.

References

[1] Personal Github. https://github.com/murcie443.

[2] Saurabh Chaudhary, Virendra Kumar, and Soumen Sen. "Design of serial link manipulator with uncertainties using interval method". In: *2020 National Conference on Emerging Trends on Sustainable Technology and Engineering Applications (NCETSTEA)*. 2020, pp. 1–6. DOI: 10.1109/NCETSTEA48365.2020.9153880.

[3] MathWorks. *Simscape Multibody*. The MathWorks, Inc. Natick, Massachusetts, United States, 2026. URL: https://www.mathworks.com/products/simscape-multibody.html.

[4] MathWorks. *Simulink*. The MathWorks, Inc. Natick, Massachusetts, United States. URL: https://www.mathworks.com/products/simulink.html.

[5] B. Siciliano et al. *Robotics: Modelling, Planning and Control*. Advanced Textbooks in Control and Signal Processing. Springer London, 2010. ISBN: 9781846286414.

[6] S. Patel and T. Sobh. "Manipulator Performance Measures – A Comprehensive Literature Survey". In: *J Intell Robot Syst* 77 (2015), pp. 547–570. DOI: 10.1007/s10846-014-0024-y.

[7] MathWorks. *gamultiobj: Find pareto front of multiple objective functions using genetic algorithm*. https://www.mathworks.com/help/gads/gamultiobj.html.

[8] The MathWorks, Inc. *paretosearch Algorithm - Global Optimization Toolbox*. URL: https://www.mathworks.com/help/gads/paretosearch-algorithm.html.